

Beyond the Syllabus

Network Flow: The 66-Year Race Toward Almost-Linear Time
Week 10 Supplement – TOAE209/TOAH209: Design and Analysis of Algorithms

Foreign Trade University

Outline

Why look beyond the syllabus?

- This week you proved that Edmonds-Karp – Ford-Fulkerson augmenting along *shortest* paths – runs in $O(VE)$ augmentations, each costing $O(E)$ for the BFS, for a total of $O(VE^2)$: polynomial, and independent of the edge capacities.
- A natural question the lecture doesn't answer: **is $O(VE^2)$ close to the best possible, or is there room for a much faster algorithm?**
- The honest answer is startling: as of this decade, max-flow can be solved in time that is **almost linear** in the size of the input – a bound that seemed out of reach for the 60+ years between Ford-Fulkerson (1956) and 2022.

How this supplement is different from a survey

We trace the *actual sequence of ideas* – not just headline numbers – from this week's $O(VE^2)$ up to $m^{1+o(1)}$, with every specific bound and year sourced to a real, checkable paper.

Why $O(VE^2)$ actually hurts at scale

Plug in graph sizes you'd meet in practice (web-scale routing, large logistics networks) and compare $O(VE^2)$ against “almost linear” ($m^{1+\epsilon}$ for tiny ϵ , where $m \approx V \approx E$ here for simplicity):

$V = E = m$	$VE^2 = m^3$	$m^{1.01}$ (illustrative “almost linear”)
10^3	10^9	$\approx 1.1 \times 10^3$
10^4	10^{12}	$\approx 1.1 \times 10^4$
10^6	10^{18}	$\approx 1.4 \times 10^6$

The gap

At $m = 10^6$, $O(VE^2) = 10^{18}$ operations is not just slow – it is *physically uncomputable* in a human lifetime on any real machine, while $m^{1.01}$ is finished before you can blink. This is the gap the algorithms below close.

From Ford-Fulkerson to almost-linear time

Year	Algorithm	Bound
1956	Ford–Fulkerson	finite, but not polynomial in n, m alone
1972	Edmonds–Karp (this week)	$O(VE^2)$
1998	Goldberg–Rao	$O\left(m \min(\sqrt{m}, n^{2/3}) \log \frac{n^2}{m} \log U\right)$
2022	Chen, Kyng, Liu, Peng, Probst Gutenberg, Sachdeva	$m^{1+o(1)}$ (randomized)
2023	van den Brand <i>et al.</i>	$m^{1+o(1)}$ (deterministic)
2025	van den Brand, Gholizadeh, Jiang, de Vos	parallel, $\tilde{O}(m+n^{1.5})$ work

For 24 years (1998–2022), Goldberg–Rao’s J.ACM bound was simply the best anyone could prove for general max-flow. Then two papers in three years broke it open.

The 2022 breakthrough

Chen, Kyng, Liu, Peng, Probst Gutenberg, Sachdeva – “Maximum Flow and Minimum-Cost Flow in Almost-Linear Time,” FOCS 2022 (**Best Paper**)

An algorithm computing *exact* max-flow and min-cost flow on directed graphs with m edges and polynomially-bounded integer capacities/costs, in $m^{1+o(1)}$ time.

- $m^{1+o(1)}$ means: for any $\epsilon > 0$, the running time is $O(m^{1+\epsilon})$ for m large enough – “almost” linear, with the hidden exponent shrinking to 0 as m grows. This is qualitatively different from any polynomial bound with a fixed exponent > 1 , like this week’s $O(VE^2)$.
- arXiv:2203.00671; published at FOCS 2022, pp. 612–623.

The core idea: replace augmenting paths with a continuous method

- Augmenting-path methods (Ford-Fulkerson, Edmonds-Karp) improve the flow *one path at a time* – inherently $\Omega(m)$ separate improvement steps in the worst case, each needing its own graph search.
- The 2022 algorithm instead builds the flow via an **interior-point method**: it takes only $m^{1/2+o(1)}$ *global* steps, and each step is a single “approximate minimum-ratio cycle” computed on the *whole* graph at once via an electrical-flow (graph Laplacian) solver.
- The remaining trick is a new **dynamic graph data structure** that maintains these approximate min-ratio cycles under edge-weight updates in *amortized* $m^{o(1)}$ *time per update* – structurally the same flavor of result as this week’s “near-constant amortized cost per operation,” just for a much harder maintained quantity than Union-Find’s connected components.

Why this result is bigger than just max-flow

The same interior-point-plus-dynamic-Laplacian-solver framework gives $m^{1+o(1)}$ -time algorithms for any flow problem minimizing a *general edge-separable convex cost* to high accuracy – not just linear costs.

The paper's own examples:

- entropy-regularized optimal transport (a core primitive in modern generative modeling and domain adaptation),
- matrix scaling,
- p -norm flows and p -norm isotonic regression on arbitrary DAGs.

The pattern to notice

A single algorithmic idea (fast dynamic Laplacian solvers) closed the gap on a 32-year-old flow bound *and* gave new best-known algorithms for four unrelated- looking problems at once – exactly how the biggest wins in algorithms tend to happen.

Why “randomized” was not the end of the story

- The 2022 algorithm is **randomized**: correct with high probability, but it uses random bits and its worst-case guarantee is over the algorithm’s own coin flips, not adversarial in the traditional deterministic sense.
- For safety-critical or adversarial settings, a **deterministic** guarantee – correct on *every* input, every time, with no dependence on luck – is strictly stronger and often required.

van den Brand, Chen, Kyng, Liu, Peng, Probst Gutenberg, Sachdeva, Sidford – “A Deterministic Almost-Linear Time Algorithm for Minimum-Cost Flow,” FOCS 2023

Derandomizes the 2022 result: exact max-flow and min-cost flow in $m^{1+o(1)}$ time, **deterministically**. This was the first improvement to the deterministic max-flow running time (for polynomially-bounded capacities) since Goldberg–Rao in 1998 – a 25-year gap.
arXiv:2309.16629.

Why “almost-linear time” still isn’t the end of the story

- $m^{1+o(1)}$ counts **total work** on a single processor. On modern hardware (multi-core CPUs, GPUs, distributed clusters), the number that determines *wall-clock* time is **depth** – the length of the longest chain of operations that must happen strictly in sequence.
- An algorithm can have optimal total work and still be slow in practice if its depth is $\Omega(m)$ – exactly the failure mode of augmenting-path methods, which are inherently sequential (each augmentation depends on the residual graph left by the last one).

van den Brand, Gholizadeh, Jiang, de Vos – “Parallel Minimum Cost Flow in Near-Linear Work and Square Root Depth for Dense Instances,” SPAA 2025

A randomized parallel min-cost-flow algorithm with $\tilde{O}(m + n^{1.5})$ total work and $\tilde{O}(\sqrt{n})$ depth, for graphs with $m > n^{1.5}$ edges – the first algorithm combining near-linear work *and* sublinear depth in that regime. Corollaries improve parallel bipartite matching, shortest paths, and reachability too. arXiv:2503.13274.

As of Aug 2026: still a very active frontier

This line of work has not slowed down. Titles accepted at ICALP 2026 continuing the same program include *“Back in the Saddle: Toward Parallel Approximate Minimum-Cost Flow”* and *“Parallel Reachability and Shortest Paths on Non-Sparse Digraphs: Near-Linear Work and Sub-Square-Root Depth”* – both pushing the 2025 parallel result further. The exact final bounds for *general* (sparse) graphs in *low* parallel depth remain open as of this writing; treat any specific number you see quoted for 2026 results as something to verify against the published version, the same discipline this supplement has followed throughout.

Summary

- 1 **This week's $O(VE^2)$** is a correct, teachable, historically accurate stepping stone – not the state of the art, and not intended to be.
- 2 **The 24-year plateau (1998–2022)** at Goldberg–Rao's bound shows that “no one has improved this in decades” is not the same claim as “no improvement exists.”
- 3 **Almost-linear (2022) → deterministic (2023) → parallel (2025)**: three separate real results were needed to turn one breakthrough into a worst-case-safe, hardware-friendly algorithm – finding the bound and making it fully deployable are different milestones.
- 4 The engine (fast dynamic graph-Laplacian solvers) is the same kind of tool this course meets again informally in Week 15, and is why the field moved past purely combinatorial augmenting-path methods for the hardest instances.

Looking ahead

Next week reduces **bipartite matching** to a single max-flow computation – in practice almost always solved with the simpler Hopcroft–Karp or Edmonds–Karp algorithms from this week, since real matching instances rarely need almost-linear-time machinery to run fast enough.